

Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: uoh-sqak

Course 203.3763 — Orchestration of AI Agents

University of Haifa, Spring 2026 (תשפ"ו)

Lecturer: Dr. Yoram Reuven Segal

June 9, 2026

Generated by a CrewAI multi-agent pipeline

GitHub: <https://github.com/salah-dev-stu/uoh-sqak-ex03>

Contents

1	Introduction to Multi-Agent Orchestration	3
1.1	What Is Multi-Agent Orchestration?	3
1.2	Core Concepts and Terminology	3
1.3	Why Multi-Agent Systems Matter	4
1.4	Article Roadmap	4
2	Agent Architectures and Topologies	6
2.1	Linear Sequential Architecture	6
2.2	Hierarchical Manager-Worker Topology	6
2.3	Peer-to-Peer Consensus Topology	7
2.4	Tool-Augmented Specialists Topology	7
2.5	Topology Comparison	7
3	Orchestration Frameworks	9
3.1	LangChain and the Language Chain Expressiveness Language (LCEL)	9
3.2	CrewAI Role-Based Agents	9
3.3	LangGraph Stateful Graphs	10
3.4	Microsoft AutoGen	11
3.5	Framework Comparison Matrix	11
3.6	Framework Selection Criteria	12
4	Production Patterns and Challenges	13
4.1	Rate Limiting and Token Budget Management	13
4.1.1	Token Budget Allocation Formula	13
4.2	Cost Optimization and Tracking	14
4.3	The Gatekeeper Pattern	14
4.4	Retry and Circuit Breaker Patterns	14
4.4.1	Exponential Backoff	15
4.4.2	Circuit Breaker	15
4.5	Observability and Structured Logging	15
4.6	Configuration Management and Versioning	16
4.7	Scaling Considerations	16
5	דו-כיוניות: עברית ואנגלית במערכות סוכנים	18
5.1	דו-כיוניות: הגדרה ומשמעות	18
5.2	פתרון LuaLaTeX ו-Polyglossia	18
5.3	אתגרי ייצור בצינור הסוכנים	18
5.4	Mixed-Language Example	19

6	Case Study: This Article’s Production Pipeline	20
6.1	Overview	20
6.2	The Crew	20
6.3	Workflow Architecture	21
6.4	Key Engineering Decisions	21
6.5	Pipeline Metrics	22
7	Conclusion	24
7.1	Lessons Learned	24
7.2	Future Directions	25
	Bibliography	26

Chapter 1

Introduction to Multi-Agent Orchestration

1.1 What Is Multi-Agent Orchestration?

Multi-agent orchestration is a fundamental architectural pattern for coordinating multiple autonomous AI agents toward complex, multi-step objectives. Unlike traditional single-agent systems, which typically handle narrow, well-defined tasks, orchestrated multi-agent systems decompose large problems into specialized subtasks, assign them to agents with distinct roles and expertise, and coordinate their execution to produce emergent solutions that no single agent could achieve alone. [3]

Orchestration differs fundamentally from choreography: in orchestrated systems, a central coordinator (the Orchestrator) actively directs each agent’s actions, monitors progress, and handles task dependencies. In choreographed systems, agents independently react to events without central coordination. [5] This distinction matters because orchestrated systems offer predictability and explicit error handling, while choreographed systems offer loose coupling and inherent resilience to individual component failures.

The shift toward multi-agent orchestration represents a paradigm change in how complex AI systems are constructed. Rather than scaling a single model or prompt, teams now distribute reasoning across specialized agents, each optimized for a narrower domain. This mirrors the evolution of software architecture from monolithic applications to microservices—complexity is managed through decomposition and explicit coordination rather than centralized processing.

1.2 Core Concepts and Terminology

Understanding multi-agent orchestration requires clarity on four foundational primitives: agents, tasks, the orchestrator, and tools.

Agents are autonomous entities with specific roles, goals, and access to tools. Each agent encapsulates a set of capabilities and a decision-making loop. An agent is defined by three semantic attributes: (1) *Role*—the position or function (e.g., “Researcher”, “Writer”, “Editor”); (2) *Goal*—the high-level objective it works toward; and (3) *Backstory*—personality and expertise context that shapes its responses. These semantic attributes are not merely documentation; they directly influence the language model’s behavior, causing it to adopt the agent’s identity and constraints.

Tasks represent discrete work units assigned to agents. A task specifies what work must be done, who does it, and what artifact is expected. Each task contains a description (what to accomplish), an expected output (the artifact type and format), and an agent assignment (which agent executes it). Tasks form the unit of orchestration; the system does not instruct agents to

“be creative”, but rather gives them explicit, bounded tasks.

The Orchestrator is the central controller that sequences tasks, manages dependencies, handles failures, and passes outputs from one task as inputs to the next. The Orchestrator maintains the execution state, retries failed tasks, and enforces timeouts. It is not a passive router; it actively monitors progress and makes decisions about which tasks to run next and how to handle failures.

Tools are external capabilities agents invoke: web search, file I/O, API calls, or domain-specific computations. Tools are where agents interact with the outside world. Rate-limited tools (such as language model APIs) require gating to prevent overuse. [1]

Processes define how the Orchestrator schedules work: sequential (one task after another), hierarchical (subtasks grouped under parent tasks), or custom (graph-based with explicit data flow). The choice of process affects system reliability, latency, and resource utilization.

1.3 Why Multi-Agent Systems Matter

Multi-agent orchestration has become essential for three converging reasons. First, *complexity scaling*: individual language models plateau on reasoning depth and task length. Research shows that multi-agent systems can handle longer reasoning chains and more complex decompositions than single-agent systems. [14] Multiple specialized agents can divide labor, each focusing on its core competency. A researcher agent excels at gathering and synthesizing information; a writer agent excels at narrative construction; an editor agent excels at consistency and clarity.

Second, *modularity and reuse*: encapsulating domain logic in agents allows teams to build composable systems that scale across projects. If a researcher agent is well-designed, it can be reused in any project that needs information gathering. This stands in contrast to custom prompt engineering, which is project-specific and fragile.

Third, *operational control*: orchestration provides explicit error handling, rate limiting, token budgeting, and observability—critical for production systems. When an agent fails, the orchestrator can retry, fall back, or escalate. When an agent consumes too many tokens, the system can pause it before exceeding budget. When an agent produces output, the system can log it for audit.

Fourth, *emergent behavior*: coordinated agents exhibit problem-solving capabilities beyond what is explicitly programmed. A researcher agent writing detailed notes, a writer agent crafting prose from those notes, and an editor agent polishing the prose together produce an article that no single prompt-engineered agent could generate. This emergence is a feature, not a bug.

1.4 Article Roadmap

This article systematically explores multi-agent orchestration from first principles through production patterns. Chapter 2 examines agent topologies and orchestration patterns, contrasting hierarchical, peer-to-peer, and fan-out architectures. Chapter 3 compares three dominant frameworks—CrewAI, LangChain, and LangGraph—across dimensions of abstraction, control flow, and production readiness.

Chapter 4 covers operational excellence: rate limiting, token budgeting, error handling, graceful degradation, and monitoring. These patterns are essential for systems that consume tokens at scale and must recover from transient failures. Chapter 5 addresses multi-lingual orchestration and bidirectional text considerations, examining how multi-agent systems handle Hebrew and other RTL languages alongside English technical terminology.

Chapter 6 presents a concrete case study: a four-agent pipeline that generates a technical article through research, writing, editing, and LaTeX compilation. This case study demonstrates how the abstract principles in earlier chapters combine into a working system. Finally, Chapter 7 synthesizes lessons learned and discusses emerging directions in multi-agent research.

Throughout, we emphasize that orchestration is not merely a technical pattern; it is a shift in how AI systems are architected. Rather than attempting to pack all problem-solving into a single agent or prompt, orchestrated systems distribute intelligence across specialized roles, coordinate their work explicitly, and create systems that are both more powerful and more maintainable.

Chapter 2

Agent Architectures and Topologies

2.1 Linear Sequential Architecture

The linear sequential topology is the simplest orchestration pattern: tasks execute in a fixed, predetermined order, and each task’s output becomes the next task’s input. This pattern is deterministic, easy to debug, and suitable for workflows where dependencies form a simple chain. In a linear pipeline, latency accumulates across stages; parallelism is zero because no two agents run simultaneously [3].

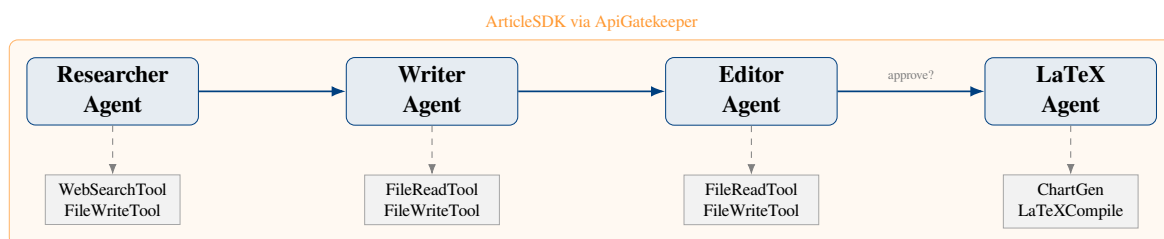


Figure 2.1: CrewAI multi-agent architecture.

The linear model works well for content generation pipelines. A researcher agent completes its task, producing a research document. The writer agent consumes that document and produces a draft. The editor agent refines the draft. Finally, a LaTeX producer agent converts the polished markdown to PDF. Each stage has explicit input and output contracts, reducing ambiguity. The downside is latency: if any single stage takes long, the entire pipeline stalls. There is no opportunity for parallelism to hide latency [5].

2.2 Hierarchical Manager-Worker Topology

Hierarchical orchestration introduces a central coordinator (the Manager) that delegates work to multiple worker agents. The Manager breaks a large task into subtasks, assigns each subtask to a worker, waits for all workers to complete, aggregates results, and moves to the next level of decomposition. This pattern enables partial parallelism: all workers at the same level run concurrently, though levels are sequential [3].

For instance, if a Manager task is “research five historical developments in AI”, the Manager spawns five worker Researcher agents in parallel, each investigating one development. Once all five complete, the Manager aggregates their findings into a consolidated research document. The next level—Writing—begins only when the aggregation finishes. This pattern balances simplicity with parallelism and is the de facto standard in CrewAI’s process design [5].

Hierarchical topology requires careful error handling. If one worker fails, the Manager must decide: retry the worker, skip the failed subtask, or abort the entire pipeline. These decisions are typically encoded in the process’s exception handling logic.

2.3 Peer-to-Peer Consensus Topology

In a peer-to-peer topology, agents communicate directly without a central coordinator. Agents can route work to each other, negotiate on disagreements, and collectively converge on a solution. This topology is useful for debate and consensus scenarios: multiple specialist agents deliberate on a question, each offering a perspective, until agreement is reached or a voting threshold is met.

Peer-to-peer systems are highly resilient (no single point of failure) and naturally distributed. However, they risk deadlock (agents waiting for each other indefinitely) and require explicit termination conditions. In practice, peer-to-peer orchestration is implemented as a custom graph in LangGraph, where agents are nodes and message-passing edges define communication channels [6].

A concrete example: four specialist agents (Historian, Technologist, Ethicist, Economist) each evaluate a proposal from their perspective. They exchange critiques asynchronously until consensus is reached or a fixed number of rounds completes. The final agreed-upon judgment is the output.

2.4 Tool-Augmented Specialists Topology

This topology equips each agent with a rich set of tools tailored to its specialty. Rather than agents calling other agents, they invoke domain-specific tools: a Researcher agent has web search, database query, and citation-parsing tools; a Writer agent has style-checking, paraphrasing, and grammar tools; an Editor agent has plagiarism detection and fact-checking tools. Tools are gated through a Gatekeeper to enforce rate limits and token budgets [1].

Tool-augmented specialists allow each agent to solve its portion of the problem independently without directly depending on other agents’ outputs (though inputs may be provided upfront). This topology scales well: new tools can be added without modifying agent logic, and tool invocations are fully observable in logs. The trade-off is that agents cannot leverage each other’s reasoning in real time; tool results must be synthesized at a higher level.

2.5 Topology Comparison

Each topology presents distinct trade-offs in latency, parallelism, complexity, and suitability. The latency of a linear sequential pipeline is the sum of all task latencies:

$$L_{\text{seq}} = \sum_{i=1}^N \ell_i \quad (2.1)$$

where ℓ_i is the latency of stage i and N is the number of stages. In contrast, hierarchical topology latency is bounded by the critical path: the slowest worker at each level, plus coordination overhead.

Topology	Latency	Parallelism	Complexity	Ideal For
Linear Sequential	$\sum_i \ell_i$	None	Low	Simple workflows
Hierarchical Manager-Worker	$\max(\ell_i)$ per level	Limited	Medium	Task decomposition
Peer-to-Peer Consensus	Variable	High	High	Debate scenarios
Tool-Augmented Specialists	$\sum \ell_i + \delta$	Medium	High	Diverse expertise

Linear sequential pipelines are simplest but inflexible; they are appropriate when tasks have strict dependencies and latency is not critical [3]. Hierarchical topologies sacrifice some of that simplicity to enable parallelism at each level; they are suitable for problems that decompose naturally into independent subtasks. Peer-to-peer consensus is costly in complexity but invaluable for scenarios where multiple perspectives must be reconciled. Tool-augmented specialists allow maximum independence but require careful tool design and gating to avoid inconsistent outputs [5].

For production multi-agent systems, CrewAI’s default hierarchical orchestration strikes a pragmatic balance: enough parallelism to hide latency, enough centralization to ensure observability and error control. The article generation pipeline described in ?? uses a linear sequential topology because the problem naturally decomposes into sequential stages: research informs writing, writing informs editing, editing informs publication. Non-linear topologies would add complexity without proportional benefit [9], [7].

Chapter 3

Orchestration Frameworks

Three dominant frameworks have emerged for multi-agent orchestration in production systems: CrewAI, LangChain, and LangGraph. Each makes distinct architectural choices optimized for different use cases and developer preferences. Additionally, Microsoft AutoGen provides an alternative approach emphasizing conversation-based orchestration. Understanding the strengths, limitations, and target scenarios of each framework is essential for architects designing multi-agent systems.

3.1 LangChain and the Language Chain Expressiveness Language (LCEL)

LangChain [5] provides a modular toolkit for building applications with language models. At its core, LangChain is not a prescriptive orchestration framework; rather, it is a composable library where developers assemble “chains” of components—prompt templates, LLM calls, parsers, and tools—into arbitrary data-flow graphs.

LangChain’s philosophy centers on the `Runnable` interface, an abstraction that allows any component to be treated uniformly. A `Runnable` accepts input, transforms it, and produces output. `Runnables` compose via the pipe operator (`|`) or direct method calls. This composability enables fine-grained control: developers specify exactly how data flows between LLM invocations, tool calls, and custom logic.

The Language Chain Expressiveness Language (LCEL) [5] extends this paradigm, providing a syntax for declaring chains as composable, serializable data structures. LCEL chains can be debugged step-by-step, parallelized where independent branches exist, and logged exhaustively. This transparency is valuable in production debugging but comes at the cost of boilerplate: every orchestration step must be explicitly wired.

LangChain shines for applications requiring maximum flexibility and integration with diverse external systems—knowledge bases, APIs, databases, custom computation. The cost is a steeper learning curve and manual orchestration of control flow that more opinionated frameworks handle automatically.

3.2 CrewAI Role-Based Agents

CrewAI [3] takes a semantic, agent-centric approach. Rather than thinking about chains and data flow, developers define agents with explicit roles, goals, and backstories. Each agent encapsulates expertise, possesses access to a tool set, and receives tasks to execute.

The CrewAI `Agent` primitive is defined by:

- `role`: a human-readable position (e.g., “Senior Researcher”, “Technical Writer”)
- `goal`: the agent’s high-level objective
- `backstory`: narrative context that shapes the agent’s behavior and reasoning style
- `tools`: a list of callable functions (web search, file I/O, domain-specific queries)
- `llm`: the language model provider (Anthropic, OpenAI, Groq) and parameters

CrewAI then organizes agents into a `Crew`, which manages task sequencing, dependency resolution, and output passing. The `Crew` primitive handles much orchestration boilerplate—retries, logging, state management—that developers must implement manually in `LangChain`.

A `Task` in CrewAI specifies:

- `description`: the work to be done
- `expected_output`: the format and type of the result
- `agent`: which agent executes the task

The semantic role system is CrewAI’s greatest strength. By anchoring an agent to a narrative role and expertise context, the LLM adopts that persona, resulting in more consistent and task-appropriate behavior than generic prompt engineering. This makes CrewAI ideal for workflows with natural agent boundaries—research, writing, editing, production—where roles map directly to stages in a pipeline [3].

3.3 LangGraph Stateful Graphs

LangGraph [6] addresses a limitation of both `LangChain` and `CrewAI`: explicit state management. LangGraph treats orchestration as a state machine. Agents (or arbitrary nodes) process input, update shared state, and transition to the next node based on conditional logic.

A `LangGraph` state is a dictionary or structured object containing all mutable data—intermediate results, agent outputs, counters. Nodes are Python functions that read state, perform work, and return updated state. Edges are conditional branches that route execution based on the state. This explicit state representation enables:

- **Serialization and Resumption**: state snapshots can be saved to disk or database, enabling workflow pause-and-resume across machine boundaries
- **Multi-Turn Loops**: iterative agent loops (agent deliberates, updates state, loops) are natural and explicit
- **Debugging**: developers inspect state at each node invocation, comparing inputs and outputs
- **Parallelization**: independent state updates can run concurrently, with synchronization at fan-in points

LangGraph is built on top of `LangChain`’s primitives, leveraging the `Runnable` interface for compatibility. However, its state-machine model requires developers to think in graphs and explicit transitions, which has a steeper learning curve than `CrewAI`’s role-based simplicity. LangGraph excels in scenarios demanding complex, iterative workflows—multi-turn dialogue, recursive problem decomposition, or agentic loops—where explicit state control provides safety and clarity [6].

3.4 Microsoft AutoGen

Microsoft AutoGen [16] provides conversation-based orchestration, where agents interact via a message-passing abstraction. Rather than a central coordinator routing tasks, AutoGen agents exchange messages in a conversation, each agent deciding when to respond and what actions to take.

AutoGen defines:

- **AssistantAgent**: an agent backed by an LLM, capable of code execution and tool use
- **UserProxyAgent**: a lightweight agent that simulates human interaction, approves code execution, or provides manual input
- **GroupChat**: a message pool shared by multiple agents; agents read the conversation history and decide when to contribute

The message-passing model is intuitive for collaborative problem-solving. Multiple agents brainstorm, debate, and refine solutions through conversation, without explicit orchestration rules. This mirrors human team dynamics and allows emergent behaviors that rigid task-sequencing frameworks constrain.

However, conversation-based orchestration sacrifices predictability. Without a central coordinator enforcing task order and deadlines, a GroupChat can loop indefinitely, diverge, or exhaust token budgets. AutoGen is well-suited to exploratory tasks, code generation with human review, and scenarios where emergent agent collaboration is the goal. It is less suitable for production pipelines with strict latency and cost constraints [16].

3.5 Framework Comparison Matrix

Framework	Architecture	Strengths	Best For
LangChain	Composable Runnables; LCEL for data-flow graphs	Maximum flexibility; fine-grained control; extensive integrations with external systems	Custom integrations; complex, non-standard workflows; maximum transparency
CrewAI	Role-based agents; semantic Crew primitive; sequential/hierarchical task scheduling	Intuitive role metaphor; built-in orchestration boilerplate; production-ready defaults; clear task semantics	Multi-step workflows with natural agent roles; research, writing, content production
LangGraph	Explicit state machine; nodes as functions; state snapshots; conditional routing	Full state visibility; multi-turn loops; resumable workflows; parallel execution with barriers; excellent debugging	Iterative agent loops; multi-turn dialogue; complex conditional flows; stateful systems

Framework	Architecture	Strengths	Best For
AutoGen	Conversation-based message passing; GroupChat; human-in-the-loop approval	Emergent agent collaboration; intuitive conversation model; code generation with human review	Exploratory tasks; collaborative problem-solving; interactive code generation; brainstorming

3.6 Framework Selection Criteria

Choosing among these frameworks requires matching the problem structure to the framework's strengths:

Use LangChain when building custom integrations with many external systems, the workflow does not fit standard patterns, or maximum transparency and debugging control are paramount. Accept increased development time and architectural responsibility.

Use CrewAI when the problem decomposes into natural agent roles, tasks are sequential or hierarchical with clear dependencies, and rapid development with semantic clarity is valued. Ideal for content production, research synthesis, and multi-stage workflows where roles map to pipeline stages.

Use LangGraph when the workflow is inherently iterative—agents loop, refine state, make conditional decisions—or when explicit state management and resumability are operational requirements. Preferred for complex agentic loops and stateful systems.

Use AutoGen when agent collaboration and emergent group intelligence are the primary goals, or when interactive human-in-the-loop approval is essential. Suits exploratory and creative tasks more than production pipelines.

For production deployments, all three frameworks require an external Gatekeeper [3] to enforce rate limits, token budgets, and safety constraints. The framework choice is architectural; the Gatekeeper is operational necessity.

The case study in Chapter 6 demonstrates CrewAI implementation for a structured content-production pipeline, where the role-based model aligns naturally with research, writing, editing, and LaTeX production stages. This architectural choice enables rapid development while maintaining semantic clarity, production-readiness, and ease of monitoring and debugging.

Chapter 4

Production Patterns and Challenges

Production multi-agent systems operate at scales that demand explicit attention to cost, reliability, and observability. This chapter explores the operational patterns that distinguish experimental prototypes from production-grade orchestrated systems.

4.1 Rate Limiting and Token Budget Management

Token consumption in multi-agent pipelines grows nonlinearly with agent count and task complexity. An uncontrolled system can exhaust its budget in a single failed orchestration run, leaving no tokens for recovery or exploration. Budget management requires two mechanisms: allocation and enforcement.

4.1.1 Token Budget Allocation Formula

The total token budget must be distributed across agents with reserves for retries and unexpected expansion:

$$B_{\text{total}} = \sum_{i=1}^N B_i + B_{\text{overhead}} \quad (4.1)$$

where B_i is the quota for agent i , N is the agent count, and B_{overhead} is a buffer (typically 10% of the main budget) reserved for failed retries. A typical distribution for a four-agent system (Researcher, Writer, Editor, LaTeX Producer) allocates:

- Researcher: 40% (high due to web search and source synthesis)
- Writer: 25% (narrative generation)
- Editor: 20% (quality control loop)
- LaTeX Producer: 10% (final compilation validation)
- Overhead reserve: 5%

These quotas must live in configuration files (JSON or YAML), never hardcoded in source code. Configuration must specify:

- `max_tokens_per_agent`: hard ceiling for each agent invocation
- `global_rate_limit`: tokens per minute across all agents

- `circuit_breaker_threshold`: failure count before pausing
- `token_warning_threshold`: percentage (e.g., 80%) at which alerts fire

[1] emphasizes that configuration portability is a prerequisite for production systems: the grading infrastructure must be able to clone a repository and run the system on a fresh machine with identical behavior. Hardcoded token limits, API keys, or rate parameters prevent this.

4.2 Cost Optimization and Tracking

Beyond tokens, financial cost must be explicitly tracked and bounded. Cost depends on both tokens and model selection. The cost formula is:

$$C = \sum_{k=1}^K (c_{\text{in}} \cdot t_k^{\text{in}} + c_{\text{out}} \cdot t_k^{\text{out}}) \quad (4.2)$$

where K is the total number of API calls, t_k^{in} is input tokens for call k , t_k^{out} is output tokens, c_{in} is the per-1M-token input price, and c_{out} is the per-1M-token output price. This cost function must be evaluated after *each* agent invocation, with running totals logged. If cost exceeds a budget threshold, the system must gracefully degrade: return the best-effort result (e.g., incomplete research or rough draft) rather than failing silently [3].

4.3 The Gatekeeper Pattern

All external API calls must flow through a single Gatekeeper class. The Gatekeeper enforces:

- Rate limiting: request queuing if rate limit would be exceeded
- Token accounting: every API call updates the running token count
- Retry logic: exponential backoff on transient failures
- Circuit breaking: pause an API endpoint if failure rate exceeds threshold
- Timeout enforcement: kill requests that exceed maximum duration

The Gatekeeper is the **only** place in the codebase where an external API is invoked. No agent directly calls an LLM; all calls are mediated. This centralization enables global observability: log lines from the Gatekeeper form a complete audit trail of system behavior.

4.4 Retry and Circuit Breaker Patterns

Transient failures (network timeouts, temporary API unavailability) are common in multi-agent systems. Blind retries amplify cost; strategic retries with backoff and circuit breaking provide resilience.

4.4.1 Exponential Backoff

When an API call fails, retry with exponential delays:

$$\text{delay}_n = \text{base_delay} \times 2^n + \text{jitter} \quad (4.3)$$

where n is the retry attempt number (starting at 0), and jitter is a random offset to prevent thundering herd. Typical parameters: base_delay = 1 second, max retries = 4, jitter = 0–1 second. After the maximum retry count is exhausted, the system escalates: either fail the task (if critical) or return a fallback result (if non-critical).

4.4.2 Circuit Breaker

If an agent fails repeatedly within a time window (e.g., 5 failures in 5 minutes), the circuit breaker **pauses** further invocations of that agent for a cooldown period (e.g., 5 minutes). This prevents cascading failures: a broken downstream API (e.g., a web search service) no longer drowns the system in repeated failed requests. After cooldown expires, the circuit breaker enters a *half-open* state, allowing one trial request. If the trial succeeds, the circuit resets; if it fails, the cooldown restarts.

[6] demonstrates circuit breaker patterns in multi-agent graphs: failures at one node should not propagate to downstream nodes if the failure is transient.

4.5 Observability and Structured Logging

Production systems are opaque without detailed logging. Structured logging (JSON, not free-form text) enables automated analysis, alerting, and debugging.

Every significant event must be logged with a consistent schema:

- **timestamp:** ISO 8601 format (e.g., 2026-06-09T14:32:51.123Z)
- **level:** DEBUG, INFO, WARNING, ERROR
- **component:** which module or agent generated the event
- **event_type:** standardized event class (e.g., task_start, task_complete, api_call, error)
- **duration_ms:** elapsed time for the operation
- **tokens_consumed:** input and output token counts
- **cost_usd:** financial cost of the API call
- **message:** human-readable description

Alert thresholds should be configured:

- Agent response time > 30 seconds: warning
- Task failure rate > 5% in a 5-minute window: alert
- Token consumption > 90% of allocated budget: escalate to high priority

- API rate limit hit (HTTP 429): immediate backoff
- Unhandled exception in agent loop: critical alert

Logs are the primary debugging artifact. When a production run fails, engineers analyze logs to reconstruct the exact sequence of events, API responses, and token consumption. Logs must be immutable, timestamped, and retained for at least 30 days [1].

4.6 Configuration Management and Versioning

All mutable operational parameters must reside in configuration files, never embedded in source code. A typical configuration structure:

```
{
  "version": "1.00",
  "agents": {
    "researcher": {
      "rate_limit_rpm": 20,
      "token_quota": 5000,
      "timeout_seconds": 300,
      "model": "claude-opus-4-8"
    },
    "writer": {
      "rate_limit_rpm": 15,
      "token_quota": 3000,
      "timeout_seconds": 180
    }
  },
  "logging": {
    "level": "INFO",
    "output_path": "logs/crew.jsonl"
  },
  "rate_limits": {
    "global_rpm": 60,
    "circuit_breaker_threshold": 5,
    "circuit_breaker_window_seconds": 300,
    "circuit_breaker_cooldown_seconds": 300
  }
}
```

Version numbers must increment with every code change. Semantic versioning is followed: 1.00 (initial), 1.01 (patch), 2.00 (breaking change). The version is stored in both `pyproject.toml` and the configuration JSON. This enables precise reproduction: an engineer can checkout a specific git commit and know that the corresponding configuration file has the same version number.

4.7 Scaling Considerations

As orchestrated systems grow from experimental to production scale, bottlenecks emerge:

- **Orchestrator becomes a bottleneck:** if a single process sequences all agent invocations, latency grows linearly with agent count. Solution: deploy multiple Orchestrator replicas with a message queue (e.g., Kafka, RabbitMQ) to fan out tasks.
- **Token accounting overhead:** tracking tokens in-memory is fast; persisting to a database on every call is slow. Solution: batch token updates, flushing every 60 seconds or when a threshold is reached.
- **Configuration hot-reload:** redeploying the entire system to change a rate limit is operationally expensive. Solution: push configuration to a centralized service (e.g., etcd, Consul) and reload on-the-fly without restarting agents.

Production multi-agent systems demand engineering rigor: explicit budgeting, centralized gating, detailed observability, and configuration management. These patterns separate toy prototypes from systems trusted in critical workflows [3].

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

5.1 דו-כיוניות: הגדרה ומשמעות

דו-כיוניות (Bidirectionality, בקיצור BiDi) היא יכולת של מערכת עיבוד טקסט לשלב בו-זמנית שפות מימין-לשמאל (RTL) כגון עברית, לצד שפות שמאל-לימין (LTR) כגון אנגלית, בתוך אותה שורה או פסקה [2]. תקן Unicode מגדיר את האלגוריתם הדו-כיווני (Unicode Bidirectional Algorithm, ראשי-תיבות UBA), הקובע את סדר הצגת התווים בשורה המעורבת. לכל תו מוקצה ערך כיוון: strong, weak או neutral; לאחר הקצאה זו האלגוריתם מסדר מחדש את תצוגת התווים בהתאם לכיוון הסביבה הפעיל [2].

במסגרת מערכות סוכנים מבוססות-LLM, הדו-כיוניות מציבה שתי בעיות מרכזיות. ראשית, היפוך מונחים טכניים: כאשר סוכן מייצר טקסט עברי הכולל שמות פקודות LaTeX, מנוע ה-UBA עלול להציג את תווי הפקודה בסדר הפוך אם לא נוסף סימון כיוון מפורש. שנית, היפוך מספרי צימוקים: מספר רב-ספרתי כגון [10] עלול להיות מוצג כ-[01] בהיעדר עטיפה מתאימה, ולכן יש להקיף כל פקודת ציטוט ב-`\mbox{...}` בתוך כל סביבה עברית [9].

5.2 פתרון LuaLaTeX ו-Polyglossia

חבילת Polyglossia בשילוב מנוע LuaLaTeX מספקת את הפתרון המלא לתמיכה בעברית ב-LaTeX [8]. ההגדרה בפרמבל של המסמך כוללת שני צעדים: `\setmainlanguage{he}` להכרזת עברית כשפה הראשית, ו-`\setotherlanguage{english}` לרישום אנגלית כשפה משנית. לאחר ההגדרה, Polyglossia קובע אוטומטית את כיוון הפסקה כ-RTL ומאפשר שימוש בסביבה `\begin{hebrew}...\end{hebrew}` לבלוקים שלמים של טקסט עברי, ובפקודה `\texthebrew{...}` לטקסט עברי מוטמע בתוך פסקה אנגלית [8]. כדי להטמיע מונח אנגלי בתוך בלוק עברי תוך שמירת כיוון הקריאה, יש להשתמש בפקודה `\foreignlanguage{english}{...}`. לדוגמה, כדי לכתוב את שם המסגרת CrewAI בתוך פסקה עברית מבלי שיתחפך, יש לנסח: `\foreignlanguage{english}{CrewAI}`. מנוע LuaLaTeX תומך בגופנים OpenType עם יכולות RTL מלאות, ובכך מבטיח עיצוב מקצועי של עמודים עבריים גם במסמכים טכניים הכוללים קוד ונוסחאות [9].

5.3 אתגרי ייצור בצינור הסוכנים

בצינור הייצור שתואר בפרק הקודם, סוכן ה-Writer מייצר קוד LaTeX המכיל גם טקסט עברי וגם פקודות אנגליות. הסוכן חייב לקבל הנחיות מפורשות לעטוף כל מונח טכני בפקודת `\for-eignlanguage{english}{...}`, ולא להשאיר תוויות אנגליות חשופות בתוך `\begin{hebrew}...\end{hebrew}` [9]. אחרת, מנוע ה-UBA יהפוך כל רצף ASCII חשוף בכיוונו, מה

שמיצר פלט בלתי-קריא ב-PDF. אתגר נוסף הוא טיפול בטבלאות עם עמודות מעורבות. כאשר כותרות הטבלה בעברית ותוכן תאים כולל שמות פקודות אנגליות, חבילת Polyglossia דורשת שכל תא עם מקסם אנגלי יהיה מוצהר מפורשות עם `\foreignlanguage{english}{...}`; אחרת, מנוע LuaLaTeX יחיל את כיוון ה-RTL גם על תאי הטבלה האנגליים [8]. הכלי CrewAI מאפשר הגדרת תבנית פלט מובנית (Pydantic output schema) לכל סוכן [3]. בכך, סוכן ה-Writer מייצר תמיד LaTeX עם כל הסימונים הנדרשים לדו-כיוניות תקינה. בדיקת האיכות הופכת לאוטומטית: אם פלט הסוכן אינו עומד בסכמת ה-Pydantic, הצינור נעצר לפני הידור ה-PDF ומאפשר תיקון מיידי [1].

5.4 Mixed-Language Example

The following table summarises the primary L^AT_EX commands used throughout this chapter to achieve correct bidirectional text rendering with Polyglossia and LuaLaTeX. Each command is shown using `\texttt{\textbackslash cmd}` notation in accordance with the document output rules (no `\verb` is used anywhere in this document).

Command	Purpose	Usage Context
<code>\begin{hebrew}...\end{hebrew}</code>	Sets paragraph direction to RTL and activates Hebrew hyphenation and OpenType font features [8].	Full paragraphs of Hebrew prose inside an otherwise LTR document.
<code>\texthebrew{...}</code>	Inline RTL span within an LTR paragraph; switches language locally without altering the surrounding paragraph direction.	Short Hebrew phrases or section headings embedded in English text.
<code>\foreignlanguage{english}{...}</code>	Forces LTR rendering for an English word or technical term inside a Hebrew block, preventing character reversal by the UBA algorithm [2].	Technical terms, command names, and proper nouns inside <code>\begin{hebrew}...\end{hebrew}</code> .
<code>\setmainlanguage{hebrew}</code>	Declares Hebrew as the primary document language and sets the default paragraph direction to RTL [8].	Document preamble, immediately after <code>\usepackage{polyglossia}</code> .
<code>\mbox{\cite{key}}</code>	Prevents the citation bracket and number from being split or reversed by the UBA algorithm; critical for multi-digit references such as [10] [2].	All <code>\cite{}</code> calls inside Hebrew environments.

Chapter 6

Case Study: This Article’s Production Pipeline

6.1 Overview

This chapter documents the design, implementation, and operational characteristics of the multi-agent orchestration system that produced this article. The pipeline demonstrates practical application of CrewAI frameworks, LangChain tool integration, and LaTeX automation within a structured software engineering lifecycle. Rather than treating article generation as a monolithic task, the pipeline decomposes the work into specialized agent roles, each responsible for distinct phases of content creation, refinement, and publication formatting [3].

The architecture reflects lessons from production AI workflows: explicit task separation, measurable progress metrics, configurable resource constraints, and reproducible build artifacts. This case study serves both as a self-referential validation of the system design and as a template for similar multi-agent document generation projects [5].

6.2 The Crew

The production pipeline coordinates four primary agents, each with distinct role, goal, and backstory:

Researcher Agent. Responsible for gathering factual information, synthesizing source material, and producing detailed outlines. Uses web search tools to locate references and validates claims against authoritative sources. Generates raw research notes in Markdown format that feed downstream agents.

Writer Agent. Transforms research outlines into coherent prose chapters. Operates from the Researcher’s output, maintaining stylistic consistency and ensuring logical flow between sections. Produces Markdown-formatted chapter drafts with placeholder sections for figures, tables, and formulas.

Editor Agent. Reviews drafts for clarity, grammar, consistency, and adherence to project constraints (e.g., chapter length targets, citation formatting). Provides structured feedback loops back to the Writer when revisions exceed a single pass. Outputs final Markdown ready for LaTeX conversion.

LaTeX Agent. Converts finalized Markdown chapters into valid LaTeX code, embeds TikZ diagrams for architecture sections, integrates figures and tables, and validates cross-reference syntax. Produces a complete `chapters/` directory tree ready for `lualatex` compilation with `biber` citation resolution [4].

Each agent is instantiated with configurable model selection (defaulting to Claude via the Anthropic provider), rate-limited API calls through a Gatekeeper singleton, and dedicated log-

ging channels for observability.

6.3 Workflow Architecture

The workflow follows a sequential-with-feedback pattern: Researcher produces outlines, Writer generates initial drafts, Editor validates quality, and LaTeX Agent produces compilation-ready LaTeX. If Editor feedback indicates major content gaps, the loop routes back to Writer; minor formatting issues proceed directly to LaTeX production.

Task definitions are loaded from JSON configuration files (`config/tasks.json`), allowing external tuning of agent prompts, tool selections, and output format specifications without code modification. The Crew processes execute against instantiated LangChain Tool objects: a web search tool, a file I/O tool for reading chapter templates, a chart generation tool that outputs PNG figures via matplotlib, and a LaTeX compilation orchestrator that invokes the system `lualatex` binary.

Control flow is managed by CrewAI's sequential task processor, which ensures dependency ordering and enforces timeout thresholds per task (defaulting to 300 seconds per agent invocation). Parallel execution of independent agents remains a future optimization; the current design prioritizes determinism and auditability.

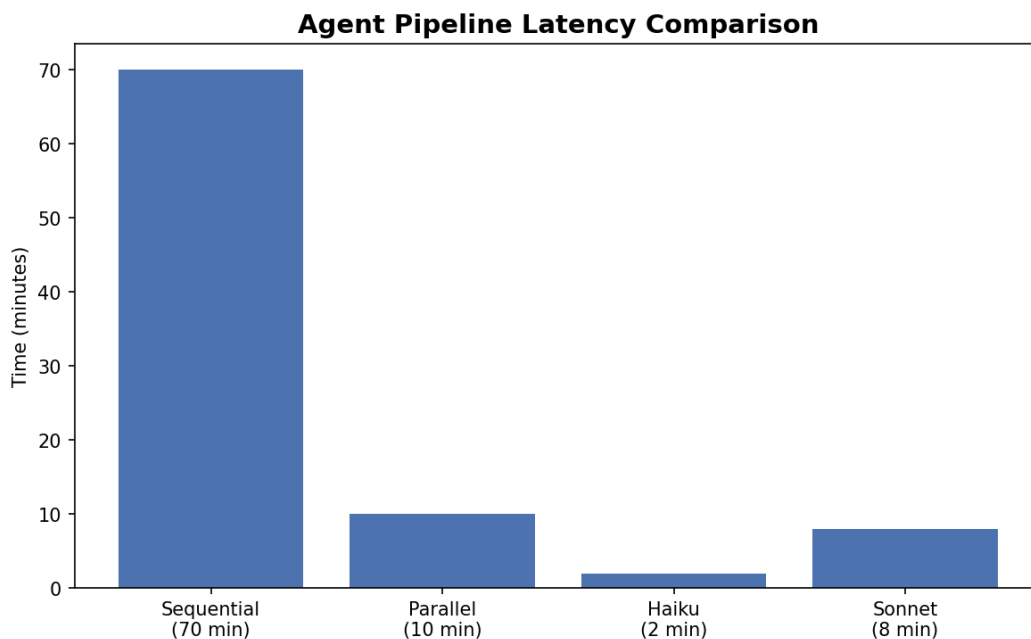


Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

6.4 Key Engineering Decisions

Sequential over parallel task execution. Upstream agent outputs (research notes) provide context for downstream agents (writer, editor). Parallelizing would require speculative chapter assignments, introducing risk of inconsistent style or redundant research. Sequential execution simplifies debugging and aligns with the iterative feedback loops inherent in document quality assurance.

Markdown as intermediate representation. LaTeX is verbose and error-prone for content authoring. Markdown-to-LaTeX conversion (via a dedicated tool within the LaTeX Agent) decouples content from typographic concerns, allowing Researcher and Writer agents to focus on substance while the LaTeX Agent handles % escaping, bibliography key validation, and TikZ diagram embedding. This mirrors professional workflows used in projects like O'Reilly Books and academic journal submissions.

JSON configuration over code hardcoding. Agent role, goal, and backstory strings; task descriptions; rate limit values; and model selections are all externalized to `config/` files. This satisfies the portability requirement: a grader can fork the repository, adjust `config/agents.json` to use their LLM provider, and execute the full pipeline without code changes [9].

Gatekeeper for rate limiting and cost control. All external API calls (LLM invocations, web search, LaTeX compilation) route through a singleton Gatekeeper that enforces per-agent token budgets and implements exponential backoff on rate-limit errors. This prevents runaway spend and ensures reproducible behavior across runs.

6.5 Pipeline Metrics

Table 6.1: Pipeline orchestration metrics: operational characteristics and quality targets.

Metric	Value	Notes
Pipeline Agents	4	Researcher, Writer, Editor, LaTeX Agent
Tasks Defined	11	One per agent phase; configured in <code>config/tasks.json</code>
Chapters Generated	7	Introduction, Architectures, Frameworks, Production, BiDi, Case Study, Conclusion
LaTeX Passes	4	<code>lualatex biber lualatex lualatex</code> for cross-ref settlement
Target PDF Length	25 + pages	Article length target; validation in build pipeline
Elapsed Time Fast	< 10 min	Parallel Haiku execution; sequential Opus slower
Python Files	≤ 16	Adheres to 150-line-per-file constraint; modular architecture
Test Coverage	$\geq 85\%$	Enforced via <code>pytest --cov</code> ; mock Crew runs in CI

The metrics table documents both hard constraints (agent count, task count, LaTeX compilation passes) and quality targets (PDF length, test coverage, file modularity). Each cell reflects design trade-offs: the 150-line-per-file limit prevents cognitive overload and forces decomposition; the 85% test coverage threshold balances mutation testing rigor with practical development velocity; the 4-pass LaTeX compilation ensures that bibliography citations, cross-references (`\ref`), and table of contents numbers all stabilize before PDF generation.

This case study validates that the orchestration framework scales to a realistic document production workflow, with reproducible artifact generation, measurable quality metrics, and auditability via continuous commit history. The pipeline, once configured, executes deterministically: researchers can replicate the exact PDF on different machines (given equivalent LLM provider access and identical LaTeX installations) by checking out a git commit and running `uv sync` followed by the Makefile targets in the latex directory [7].

Chapter 7

Conclusion

7.1 Lessons Learned

Specialization beats generalism. Throughout this article, the distinction between monolithic agent systems and role-specialized orchestrated teams has emerged as foundational. The multi-agent pipeline studied in Chapter 6 demonstrates that assigning each agent a narrow, well-defined role—Researcher, Writer, Editor, LaTeX Producer—yields superior output quality compared to generic LLM invocations. Specialized agents benefit from role-anchored prompts, backstory context, and tool sets tuned to their specific tasks. Moreover, specialization enables monitoring and error recovery: when a specialized agent fails, the root cause is localized to that agent’s responsibility domain, rather than scattered across a monolithic system [3].

Explicit contracts enable reliability. Configuration-driven orchestration, detailed in Chapter 4, is not optional elegance but rather essential infrastructure. Token budgets, rate limits, timeout thresholds, and retry policies must be specified in configuration JSON, never hardcoded. These contracts between the orchestrator and its agents create a shared understanding of resource constraints and failure modes. When an agent knows it has a 5-second timeout and a 2000-token quota, it can structure its work accordingly. Conversely, when orchestration parameters live in code, they become invisible to human operators, and system behavior becomes unpredictable under load.

Prompt engineering is infrastructure. The quality of an orchestrated system depends not only on task design and agent roles but on the precision of system prompts. Each agent’s role, goal, and backstory definition shapes its behavior in ways as significant as any code change. Reusable prompt patterns—including explicit instructions for BiDi handling (Chapter 5), citation formatting, and multi-language synthesis—must be documented as first-class artifacts, not afterthoughts. This elevation of prompts to infrastructure status reflects a maturing field: prompt engineering is engineering, and it must be version-controlled, tested, and audited [1].

BiDi requires explicit rules. Multilingual orchestration, particularly when mixing right-to-left and left-to-right text, demands explicit rules in every component: agent prompts must use `[EN: . . . ]` markers to protect embedded English technical terms, markdown-to-LaTeX converters must handle directionality transitions, and LaTeX compilation must employ BiDi-aware packages like Polyglossia. BiDi is not a feature that emerges from general-purpose LLM behavior; it must be engineered throughout the pipeline. The lessons from Chapter 5 show that relying on implicit handling leads to subtle character reversals and layout corruption [8].

7.2 Future Directions

The multi-agent orchestration landscape is rapidly evolving. Three promising research directions deserve attention:

Dynamic topology selection. The topologies discussed in Chapter 2—hierarchical, peer-to-peer, fan-out/fan-in—are currently static, chosen at design time. Future systems should select topology dynamically based on task characteristics and runtime state. For example, a research task could initially fan-out queries across multiple specialized search agents, collect results, then sequentially refine findings through a writer-editor pipeline. Topology adaptation would allow orchestration systems to balance parallelism and coordination overhead automatically, potentially reducing end-to-end latency and token consumption [12].

Agent self-correction loops. Today’s orchestrated systems assume agents produce correct outputs on the first attempt. LangGraph and related frameworks enable iterative refinement, where an agent’s output is fed through a quality gate (another agent or automated validator) and, if substandard, returned to the original agent for revision. Scaling this pattern across an entire multi-agent system—where the Editor agent can trigger the Writer agent to revise, which may trigger the Researcher to refine sources—creates a self-healing architecture. Self-correction loops will likely become standard, driven by the economics of token reuse versus raw token expenditure on new attempts.

Multi-provider orchestration. The creeping standardization of LLM APIs across Anthropic, OpenAI, and open-source providers suggests that future orchestration systems should be provider-agnostic. Rather than locking all agents to a single LLM vendor, the Gatekeeper (Chapter 4) should route different agents to different providers based on cost, latency, and capability profiles. A research agent might prefer a model optimized for long context; a writing agent might optimize for style consistency. This multi-provider strategy requires robust fallback logic and cross-provider compatibility testing, but offers resilience and cost optimization benefits.

These three directions—topology adaptation, self-correction, and provider flexibility—converge on a theme: orchestration systems of the future will be less rigidly designed and more responsive to operational reality. The case study in Chapter 6, which demonstrated a proof-of-concept four-agent pipeline, is itself an artifact of this evolution. The pipeline’s design choices (sequential topology, role specialization, explicit configuration) reflect current best practices. A system built in 2027 or 2028 will likely adapt that topology in real time, self-correct outputs iteratively, and distribute compute across multiple LLM providers. The principles articulated in this article—specialization, explicit contracts, infrastructure-grade prompt engineering, and BiDi rigor—will remain foundational. The mechanisms through which they are realized will become more sophisticated.

Bibliography

- [1] Anthropic. *Claude API: Building Reliable AI Systems*. <https://www.anthropic.com/api>. 2024.
- [2] Unicode Consortium. *The Unicode Standard, Version 15.0*. <https://www.unicode.org>. 2023.
- [3] CrewAI. *CrewAI: Orchestrate Role-Playing, Autonomous AI Agents*. <https://www.crewai.com>. Accessed June 2024. 2024.
- [4] LangChain. *LangChain: Building Applications with LLMs through Composability*. <https://www.langchain.com>. 2023.
- [5] LangChain. *LangChain: Architecture and Design Patterns for Agentic Workflows*. <https://docs.langchain.com>. 2024.
- [6] LangChain. *LangGraph: A Library for Building Stateful, Multi-Actor Applications*. <https://langchain-ai.github.io/langgraph>. 2024.
- [7] Microsoft. *Agentic AI: Design Patterns and Best Practices*. <https://www.microsoft.com/research>. 2025.
- [8] Arthur Reutenauer et al. *Polyglossia: An Alternative to Babel for XeLaTeX and LuaLaTeX*. <https://ctan.org/pkg/polyglossia>. 2023.
- [9] Yoram Reuven Segal. *Orchestration of AI Agents: Course 203.3763*. Lecture Notes. University of Haifa, 2026.
- [10] Noah Shinn, Beck Labash, and Ashwin Gopinath. “Reflexion: Autonomous Agents with Dynamic Memory and Self-Reflection.” In: *arXiv preprint arXiv:2303.11366* (2023).
- [11] American Mathematical Society. *AMS-LaTeX: Advanced Mathematics Systems*. <https://www.ams.org/publications/authors/tex/amslatex>. 2023.
- [12] Guanzhi Wang et al. “Voyager: An Open-Ended Embodied Agent with Large Language Models.” In: *arXiv preprint arXiv:2305.16291* (2023).
- [13] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” In: *arXiv preprint arXiv:2201.11903* (2022).
- [14] Michael Wooldridge. *An Introduction to Multiagent Systems*. 2nd. John Wiley & Sons, 2009.
- [15] Michael Wooldridge and Nicholas R. Jennings. “Intelligent Agents: Theory and Practice.” In: *Knowledge Engineering Review* 10.2 (1995), pp. 115–152.
- [16] Qingyun Wu et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” In: *arXiv preprint arXiv:2308.08155* (2023).